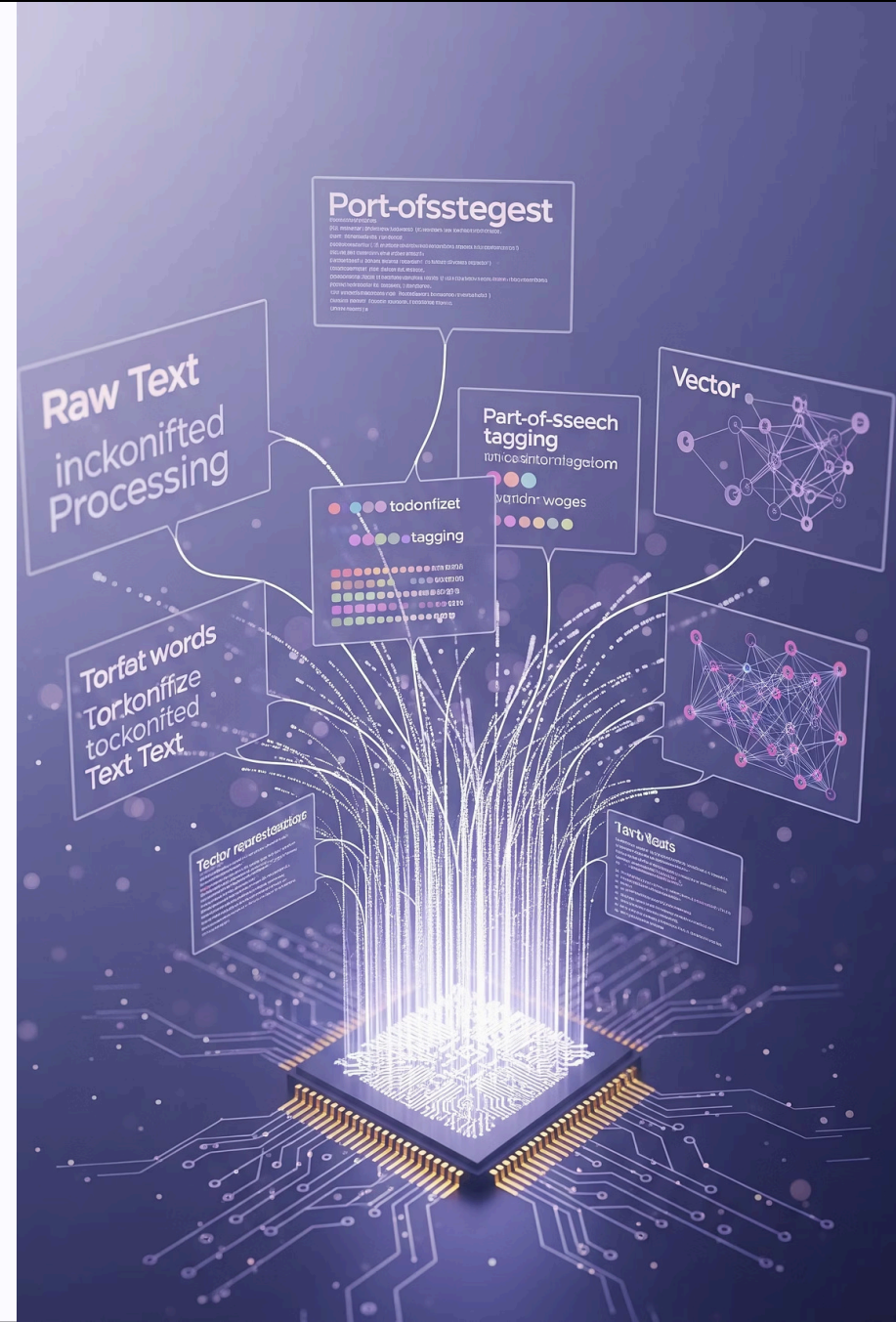


Efficient Text Pipelines for Training & Inference



Why NLP Pipelines Matter

Text is inherently messy and challenging to process efficiently:

- Unicode variations and normalization challenges
- Inconsistent casing and formatting
- Special content like emojis and code blocks
- Throughput bottlenecks at **tokenization and batching** stages



Well-designed pipelines enable **semantic search, clustering**, and state-of-the-art performance across language tasks.

Tokenization Methods

ac laars
pins
the
the
aber
terigizaten
nardnanstire
fvik
thel
fom(roseltin)
lleok
toornentior

tokeniztaion
cand
sam(ibi)
sext(irestiond)
rompriorn
heercel
beud
neang
tickinization
spontig
liek(alresgion)

Tokenization Basics

Whitespace/Word Tokenization

Simple but brittle approach that breaks on unseen words

"I love NLProc!" → ["I", "love", "NLProc!"]

Subword Tokenization

Modern approach using BPE, WordPiece, or Unigram methods

"I love NLProc!" → ["I", "love", "NL", "Proc", "!", ""]

Out-of-Vocabulary Handling

Decomposes unknown words into known subword units

Trade-off between vocabulary size and sequence length

Practical Tokenization Tips

Preprocessing

- Normalize Unicode characters consistently
- Apply lowercase based on task requirements
- Handle special tokens for model-specific needs

Performance Optimization

- Use **Rust-backed fast tokenizers** (HuggingFace)
- Batch inputs for maximum throughput
- Profile tokenization as a potential bottleneck

Sequence Management

- Pad/truncate to maximum length (typically 512-2048 tokens)
- Avoid excessive padding that wastes compute
- Consider sliding window for long documents

Versioning

- Always maintain tokenizer version alongside model artifacts
- Test backward compatibility when upgrading

Embeddings 101

What Are Embeddings?

Dense vector representations of text that capture semantic meaning

- Fixed-dimension floating point vectors (128-1536)
- Measure similarity via cosine / dot product / L2
- Enable semantic matching beyond keyword search



Common Use Cases

Retrieval-Augmented Generation (RAG)

Fetch relevant documents to ground LLM responses

Semantic Search

Find conceptually similar content beyond keywords

Clustering & Deduplication

Group similar documents and remove duplicates

Recommendation

Suggest related content based on semantic similarity



Building Embedding Pipelines



Optimize Throughput

Batch inputs aggressively for GPU/TPU efficiency

Pre-compute embeddings for static content



Store & Index

Use specialized vector databases: **FAISS**, **Pinecone**, **Weaviate**

Store metadata alongside vectors for filtering



Balance Tradeoffs

Choose dimension size based on accuracy/latency needs

Consider model size vs. quality tradeoffs

Remember to refresh embeddings when data changes and monitor for embedding drift and quality degradation over time.

Transformers at a Glance

Encoder-only (BERT)	Decoder-only (GPT)	Encoder-decoder (T5)
Bidirectional context	Unidirectional (left-to-right)	Full sequence transformation
Classification & encoding	Text generation	Sequence-to-sequence tasks
Ex: sentiment, NER, embedding	Ex: completion, chat, QA	Ex: translation, summarization

The Attention Mechanism

Core innovation that enables:

- Learning contextual relationships between words
- Capturing long-range dependencies in text
- Parallel processing of sequence elements
- Self-attention: $O(n^2)$ complexity with sequence length



Throughput & Latency Levers



Sequence Length

Long sequences increase memory and compute: $O(n^2)$ attention complexity



Optimization Techniques

Implement **Flash Attention**, mixed precision (BF16/FP16)



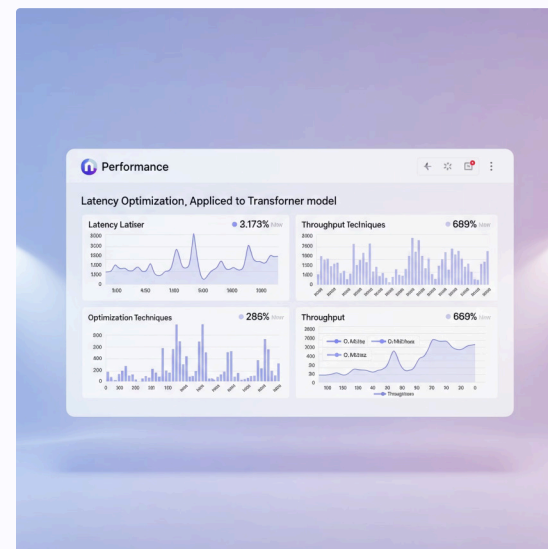
Efficient Batching

Dynamic batching + KV-cache for generation tasks



Quantization

INT8/INT4 for edge deployment and high QPS services



Benchmarking is essential - performance characteristics vary widely across hardware platforms and model architectures.

Training & Serving Patterns

1 Training Workflow

Pretrain on large corpus → Fine-tune/PEFT (LoRA, adapters) for specific tasks

Compute once, deploy many times

2 Retrieval Pattern

Embed query → Search vector index → Rerank candidates → Present answers

Optimize for recall in early stages, precision in later stages

3 Generation Pattern

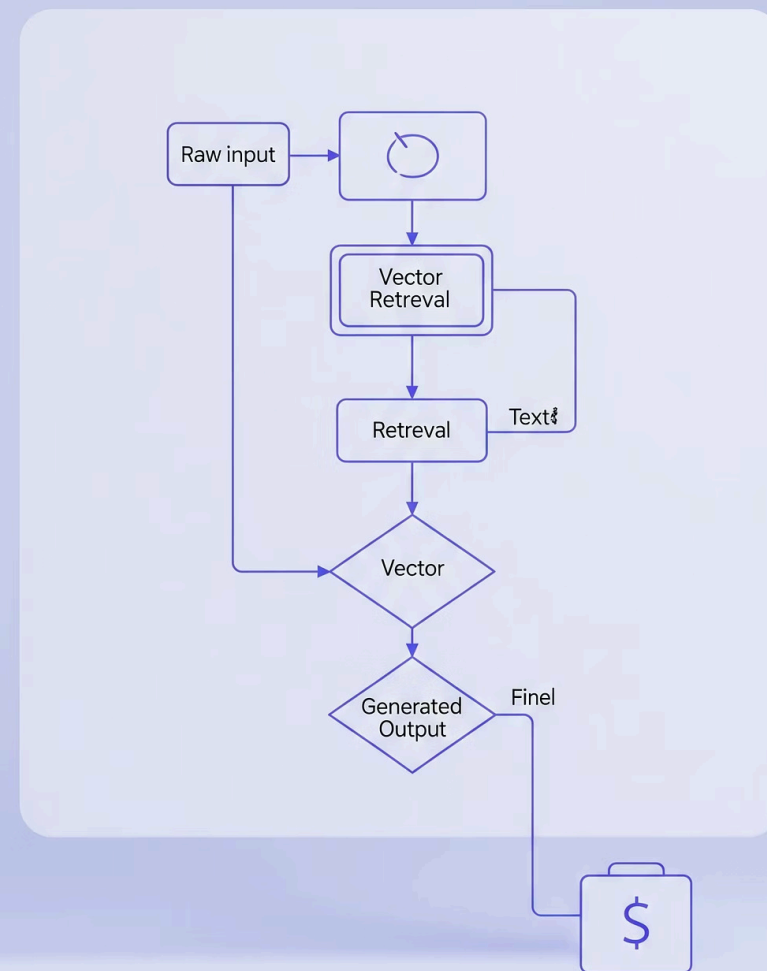
Process prompt → Retrieve context (optional RAG) → Decode tokens → Stream response

Optimize for responsiveness and coherence

4 Optimization Tips

Cache hot prompts, embedding results, and KV states

Use speculative decoding for faster generation



Quality & Evaluation

1

Classification Metrics

- Accuracy, Precision, Recall
- F1 Score (harmonic mean)
- ROC-AUC (ranking quality)
- Confusion matrix analysis

2

Generation Metrics

- BLEU, ROUGE (n-gram overlap)
- METEOR (synonym matching)
- BERTScore (semantic similarity)
- Perplexity (prediction confidence)

3

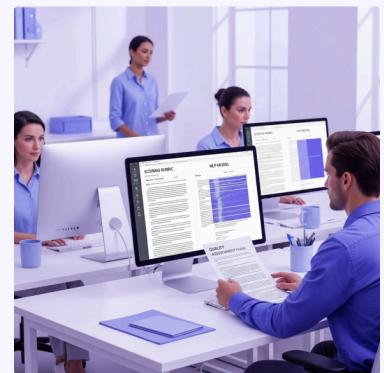
Retrieval Metrics

- Recall@k (found relevant items)
- Mean Reciprocal Rank (MRR)
- Normalized Discounted Cumulative Gain (nDCG)
- Latency-at-percentile (p95, p99)

Human Evaluation

Essential complement to automated metrics:

- Factual accuracy and hallucination detection
- Safety and bias assessment
- Writing style and tone appropriateness
- Overall usefulness for intended application



Best Practices



Version Management

Keep **tokenizer, model, and embeddings** in lockstep versions

Test backward compatibility before deployment



Monitoring

Track input length distribution and cap extremes

Monitor recall vs. latency in vector search operations



Guardrails

Implement profanity filters and PII detection

Create robust prompt templates with constraints

Common Pitfalls

- Tokenizer mismatch between training and inference
- Embedding model drift causing retrieval degradation
- Unconstrained input lengths creating tail latency
- Excessive padding wasting computational resources



Key Takeaways

Building efficient NLP pipelines requires optimization at every stage

Tokenization Efficiency

The foundation of NLP throughput

Use fast tokenizers and consistent preprocessing

Embeddings

Enable semantic operations at scale

Vector search unlocks retrieval capabilities

Transformers

Power state-of-the-art NLP across domains

Choose architecture based on task requirements

Technical Optimization Levers:

Maximize efficiency with **batching**, **KV-cache**, **quantization**, and **Flash Attention**

Questions? Contact: nlp-infrastructure@company.com